

User-space Paging on seL4

September 21, 2026

Fast memory paging is essential for general-purpose “emerging applications” such as those on smartphones, where cases like app startup rely on fast fault handling to reduce startup latency for the user. Chen et al. [2024] claim that microkernels are inherently disadvantaged when servicing a page fault. On a microkernel, the fault must be redirected to the memory pager server, which uses a kernel invocation to map the page to a memory frame, using a capability to the frame to do so. In contrast, on a monolithic kernel, the fault can be directly serviced by the kernel. Chen et al. [2024] make the claim that a “significant performance degradation [...] due to the slow paging procedure of anonymous memory” is observed, that is “primarily due to the extra round-trip from the kernel to the pager”. A secondary claim is that “capabilities [...] introduce significant performance overhead due to the frequent updating of some kernel objects (e.g., the page table) managed outside the kernel and inhibit efficient cooperation between them [...] causing the handling of anonymous page faults 3.4× slower than Linux”.

Due to this, the pager has been moved inside the kernel in HongMeng, eliminating the extra round-trip IPC. The kernel pager, on a fault, directly maps a pre-allocated memory frame to the faulting page and then records an operations log for the user-space memory manager server to implement policies [Chen et al., 2024]. This departs from Liedtke’s minimality principle [Liedtke, 1995], and we believe that their justification for this design choice is not sufficiently substantiated.

1 HongMeng benchmarks

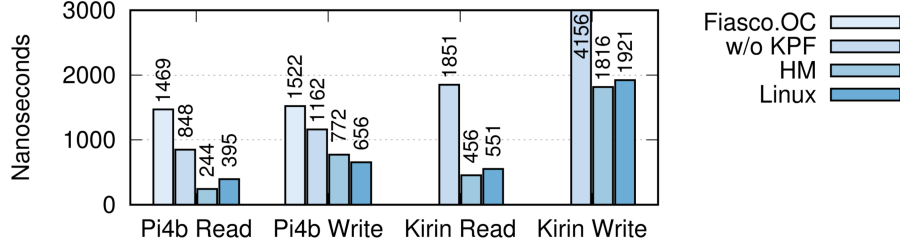


Figure 1: Page fault latency of private anonymous memory: HongMeng.

w/o KPF denotes the HongMeng kernel with a user-space pager.

This benchmark involved mapping anonymous memory via *mmap* without pre-populating, then performing page-by-page operations to trigger page faults on first access. The benchmark measured average latency, calculated by triggering a batch of page faults and measuring the total elapsed time, and dividing it by the number of pages. Source: [Chen et al., 2024]

Figure 1 shows the performance improvement of page fault latency that Chen et al. [2024] achieved in the HongMeng kernel by moving the pager into the kernel. They record a latency reduction of 72%/33% on Pi4b and 75%/55% on Kirin9000 (little core). The Linux versions used for comparison here are 6.1 for Pi4b and 5.10 for Kirin.

2 Argument for user-space paging

The argument is that the IPC cost and frame capability invocation (from mapping) are not as significant a cause of performance degradation as claimed by Chen et al. [2024]. It should be possible to implement performant (similar to Linux) user-space paging without compromising on a principled microkernel design. Thus, a user-space pager on LionsOS (which is based on seL4) should demonstrate that private anonymous page fault latency similar to that of Linux can be achieved on a traditional (unmodified) microkernel. We do this by replicating the private anonymous page fault latency experiment of Chen et al. [2024], depicted in Figure 8 of “Microkernel Goes General: Performance and Compatibility in the HongMeng Production Microkernel.”

3 Our benchmarking

Minor page fault latency

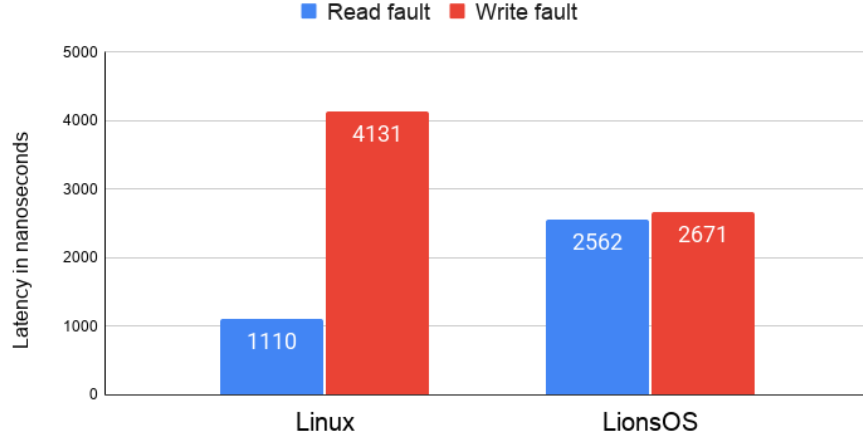


Figure 2: Page fault latency of private anonymous memory: LionsOS. Measured on Pi4b with a 1.5GHz clock rate. Linux version is 6.12.61-v8. Read is optimised with zero page.

Figure 2 shows our benchmarking of page fault latency which follows the same methodology as in Figure 1. This benchmark shows the average page fault latency measured by triggering a batch of page faults and measuring the total elapsed time divided by the number of pages. This was run for 16384 4096 byte pages on Pi4b running at 1.5GHz clock frequency. The page faults were triggered by accessing private anonymous, not pre-populated memory, mapped via *mmap*.

3.1 Write fault

The general process to service write faults is:

1. Get a (pre-allocated) frame cap to an unused memory frame.
2. Create shadow page tables to keep track of this frame.
3. Map (pre-allocated) intermediary paging structures.
4. Map frame cap to faulting page.
5. Resume execution on faulting PD

3.2 Read fault

The `mmap()` syscall allocates zero-filled pages, hence the global zero-page optimisation is triggered for Linux, requiring less bookkeeping as the global zero page can simply be read-only mapped. The LionsOS pager also implements the same optimisation, mapping a global zero frame. However, as the same frame cannot be mapped into the same vspace using the same capability, a pre-allocated capability copy must be used to do the mapping, and as we cannot lose track of the capability (as we do not want to leak CNode slots), we must also create shadow page tables for this capability copy and map this frame to the faulting page. This results in a similar amount of bookkeeping as write faults and the same number of kernel invocations (mapping intermediary paging structures and page mapping) resulting in a similar latency. Figure 1 shows that this affects Fiasco.OC similarly as they also use a similar capability model and therefore read and write fault latencies are not significantly different.

Figure 2 shows an approximate average latency of 2600ns per fault for both write and read faults. The `PAGE_MAP` operation accounts for approximately 847-885ns (read, write). Approximately 200-300ns are used for bookkeeping and the rest is accounted for by the fault delivery and round trip IPC. The pager is implemented as a passive server to reduce scheduling overhead.

4 Conclusion

We observe from Figure 2 that write fault handling can be done with a lower latency than Linux, despite the inclusion of the extra round-trip IPC and capability operations. The global zero-page optimisation is approximately $2.5\times$ slower than Linux, but this is an edge case optimisation that only occurs on reads to zeroed out pages or previously unaccessed pages. This shows that the argument for introducing the pager into the kernel to achieve better performance is not so obvious. In order to keep the option of formal verification for the kernel open, the pager should be implemented as a user-space server.

5 Weird results from Figure 1

I failed to match the measurements from Chen et al. [2024], and here are some weird things I have noticed that confuse me:

- Kirin 9000 (little core) which is a cortex a55 at 2.05GHz is drastically slower than Pi4b which is a cortex a72 at 1.5GHz.
- Fiasco.OC achieves around 1500ns latency per fault. This is 2250 cycles to do the IPC and fault delivery, mapping operations and bookkeeping. Given that their fastpath IPC is slower than seL4's, this seems implausible.
- Chen et al. [2024] claims for seL4 that “the round-trip (to the pager) of fault handling takes about 140ns on Pi4b (measured using sel4bench),

which makes it significantly slower than Linux.” This is a strange claim to me as a round-trip IPC fastpath on seL4 on Armv8a takes around 1000 cycles, not the 210 cycles claimed here.

References

- Haibo Chen, Xie Miao, Ning Jia, Nan Wang, Yu Li, Nian Liu, Yutao Liu, Fei Wang, Qiang Huang, Kun Li, Hongyang Yang, Hui Wang, Jie Yin, Yu Peng, , and Fengwei Xu. Microkernel goes general: Performance and compatibility in the HongMeng production microkernel. In *USENIX Symposium on Operating Systems Design and Implementation*, pages 465–485, Santa Clara, CA, US, July 2024. USENIX.
- Jochen Liedtke. On μ -kernel construction. In *ACM Symposium on Operating Systems Principles*, pages 237–250, Copper Mountain, CO, USA, December 1995. ACM.